SOFTWARE SYSTEMS DEVELOPMENT

FastAPI & Modern Web Systems

Concurrency models, ASGI protocols, Pydantic schemas, Jinja templating, and JWT security pipelines.

FOUNDATIONAL SYSTEMS THEORY

Concurrency vs. Parallelism

Concurrency (Interleaving)

Dealing with many things: Single CPU switches tasks.

The Solo Chef: Chop vegetables while soup simmers.

FastAPI Model: Event loop awaits non-blocking I/O.

Parallelism (Simultaneous)

Doing many things: Multi-core physical execution.

Multiple Chefs: Two people cook two pots at once.

Deployment Model: Multiple Uvicorn OS worker processes.

EXECUTION PROTOCOLS

WSGI vs. ASGI: The Blocking Difference

WSGI (Flask / Django): Blocking

```python
import time
from flask import Flask

@app.route("/slow")
def slow():
  time.sleep(5) # Freezes worker
  return "Done"
```

Worker thread stays locked for 5 seconds.

ASGI (FastAPI): Non-Blocking

```python
import asyncio
from fastapi import FastAPI

@app.get("/fast")
async def fast():
  await asyncio.sleep(5) # Yields control
  return {"message": "Done"}
```

Event loop handles concurrent requests during sleep.

INTERACTIVE WALKTHROUGH 1

Live Server Execution & Terminal Diagnostics

```python
# activity1.py: Minimal ASGI Entry
from fastapi import FastAPI
app = FastAPI()

@app.get("/")
def read():
  return {"message": "my first fastapi server"}
```

Run Server with Hot-Reload

```
uvicorn activity1:app --reload
```

Troubleshoot Port 8000 Clashes

```
lsof -i :8000
kill -9 (Never kill -9 1)
```

ROUTING SYSTEM

Routes: The Server's Delivery Addresses

API: The **entire contract/interface** that defines how two systems talk to each other. It includes all routes, request formats, data schemas, authentication, and responses.

```
# Declarative Route Definitions
@app.get("/api/v1/users")
def list_users():
  return [{"name": "ABC"}]

@app.post("/api/v1/users")
def create_user(user: User):
  return user
```

Postal Address Analogy

Every URL endpoint acts as a postal destination with an associated function handler.
Endpoint: The **specific target location** (URL + HTTP verb) where an external client accesses a resource on an API.

RESTful API maps CRUD to HTTP methods

Clean REST verb semantics: GET for reading, POST for creating, and DELETE for removing, PUT for updating

API DESIGN PATTERNS

Path Parameters vs. Query Parameters

Path Parameter: /items/{id}

Identifies Resource: Direct resource key.

Mandatory: Route fails without parameter.

Example: /users/42

Query Parameter: ?skip=0&limit=10

Filters Resource: Pagination, filter or search.

Optional Defaults: Assigned in signature.

Example: /items?category=books

URL PARAMETERS

Path Parameters: Dynamic URL Capture

```
# Type-Annotated Path Parameter
@app.get("/todos/{todo_id}")
def get_todo(todo_id: int):
  return {"todo_id": todo_id}

# Non-Integer Rejection (Automatic)
GET /todos/abc → HTTP 422
```

Automatic Type Parsing

Incoming URL segments automatically cast to Python integers prior to function invocation.

Automatic 422 Error Handling

Invalid types generate standardized JSON validation responses without boilerplate error code.

CORE DATA VALIDATION

Pydantic

1. Order Checklist

Strict type specification for incoming fields.

Numerical integer quantity

String descriptor text

Boolean state flag

2. Smart Correction

Automatic type coercion before raising errors.

String to Int: "2" coerced to 2

Truthiness: "true" coerced to True

Rejection: "rocks" raises error

3. Canonical Notebook

Guaranteed schema compliance downstream.

Predictable: Zero runtime KeyErrors

IDE Ready: Type hinting autocomplete

Export: Native JSON conversion

INTERACTIVE WALKTHROUGH 2

Pydantic Schemas & Validation Cases

```python
class User(BaseModel):
  id: int
  name: str
  age: Optional[int] = None

# Parsing with keyword unpacking
User(**user_data)
```

Case 1: Full payload
`{"id": 1, "name": "ABC", "age": 21}` → Valid

Case 2: Optional field omitted
`{"id": 1, "name": "ABC"}` → age defaults to None

Case 3: Type failure
`{"id": "abc"}` → Raises ValidationError

INTERACTIVE WALKTHROUGH 3

How Decorators Power FastAPI Routes

```
# Higher-Order Function Pattern
def decorator_function(func):
  def wrapper():
    print("Before execution")
    func()
    print("After execution")
  return wrapper
```

The Cake Analogy

The base function runs core logic; decorators add icing without altering underlying code.
Maintains Separation Concerns, Dont Repeat Yourself (DRY)

FastAPI Application

Syntactically, it is represented by the @ symbol placed directly above a function definition (like @app.get("/users") or @login_required).
Eg⇒ writing `@my_logger` above `calculate_total` `is equivalent to calculate_total = my_logger(calculate_total)`

INTERACTIVE WALKTHROUGH 4

Swagger UI & In-Memory CRUD Endpoints

Interactive UI: /docs#/

Execute POST, GET, PUT, and DELETE operations directly from browser with zero Postman setup.

Auto-Generated Schemas

Reflects Item, ValidationError, and HTTPValidationError models automatically.

```
# Path Parameter CRUD Pattern
@app.get("/items/{item_id}", response_model=Item)
def get_item(item_id: int):
  for item in items_db:
    if item['id'] == item_id: return item
  return {"Item not found"}
```

FULLSTACK INTEGRATION

Jinja2 Templating & Static Assets

```python
# Mounting Statics & Templates
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates

app.mount("/static", StaticFiles(directory="static"))
templates = Jinja2Templates(directory="templates")

@app.get("/render")
def home(request: Request):
  return templates.TemplateResponse("base.html",
  {"request": request})
```

Expression Syntax: {{ ... }}

Interpolates server variables and computed expressions directly.

Control Blocks: {% ... %}

Manages conditionals, loops, and template inheritance blocks.

NETWORK SECURITY & BROWSERS

CORS: Cross-Origin Resource Sharing

```python
# Enabling Frontend Access

from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
  CORSMiddleware,
  allow_origins=["http://localhost:3000"],
  allow_credentials=True,
  allow_methods=["*"],
  allow_headers=["*"],
)
```

The Problem: Origin Mismatch

Browsers block JS on port 3000 from reading data from port 8000 under the Same-Origin Policy.

The Solution: Preflight Headers

Middleware handles HTTP OPTIONS preflights and returns Access-Control-Allow-Origin headers.
Configured on backend

SECURITY ARCHITECTURE

Stateless Auth: Sign & Decode JWTs

```
# Signing Claims with Secret
def sign_jwt(user_id: str) → Dict[str, str]:
  payload = {"user_id": user_id, "expires": time.time() +
  600}
  token = jwt.encode(payload, JWT_SECRET,
  algorithm=JWT_ALGORITHM)
  return {"access_token": token}
```

```
# Decoding and Expiry Checks
def decode_jwt(token: str) → dict:
  try:
    decoded = jwt.decode(token, JWT_SECRET,
    algorithms=[JWT_ALGORITHM])
    return decoded if decoded["expires"] ⩾ time.time()
    else {}
  except:
    return {}
```

DEPENDENCY INJECTION GUARDS

Declarative Route Protection with HTTPBearer

```python
# Guarding Routes Declaratively
@app.post("/posts", dependencies=[Depends(JWTBearer())])
async def add_post(post: PostSchema):
  posts.append(post.dict())
  return {"data": "post added securely"}
```

Automatic 403 Forbidden

Missing headers or invalid schemes reject requests before hitting the endpoint function.

Clean Separation of Concerns

Authentication lives completely inside the dependency injection layer, keeping handlers clean.

SYSTEMS LAB SUMMARY

FastAPI Architecture to Production

Non-blocking ASGI execution, strict Pydantic schemas, dynamic routing, and stateless JWT authorization.